

An Exercise for i-Vis Engineering Position

For i-Vis Lab Use Only

This exercise aims to test software engineers applying to i-Vis Labs. It is for developing a small web application to perform some simple queries on a movie dataset stored in [Neo4j](#) graph database at backend and visualize the results using [Cytoscape.js](#) graph visualization library at frontend. You can use the default movie dataset in Neo4j whose details are given [here](#).

Use Cases

The application is to mainly display a neighborhood of a given actor or a specific movie to visually analyze the contents of the movie dataset. The initial query to the database will be with the neighborhood of a given actor but the user will also be able to gradually extend the displayed sub-network with additional queries to the Neo4j movie dataset.

Overall GUI

Your browser based application is expected to have a simple user interface as follows. The application web page will contain necessary menu/buttons to accept an actor name and a positive integer (actor number) as input and will have a visualization screen (a Cytoscape.js canvas) to show immediate neighborhood of the given actor up to the provided integer, composed of actors and movies with their relationships.

“Actor number” of an actor is the number of degrees of separation s/he has from a given actor. For example, suppose the given actor is “Tom Hanks”. “Tom Hanks”'s actor number is 0. The actors who played with “Tom Hanks” in the same movie (such as “Leonardo DiCaprio” who played in “Catch Me If You Can” with “Tom Hanks”) has actor number of 1. The actors who played with the actors who have an actor number of 1 in the same movie (such as Kate Winslet who played in “Titanic with Leonardo Di Caprio”) has actor number of 2 and so on.

Given an actor name and a positive integer as input, your application should find the subgraph that consists of actors that has actor number of at most the given integer and movies that cause those actor numbers by accessing to and making required queries on the movie dataset stored in Neo4j. Then, it should display the resulting graph to the user by using Cytoscape.js graph visualization library. Since nodes of this graph do not have any layout information, you should perform an automatic layout on it before presenting it to the user. For this purpose, use the fCoSE layout algorithm, implemented as a Cytoscape.js extension.

Node/Edge UI and Extending the Graph

In the resulting graph, actors and movies will be represented with nodes with distinct images (choose your own). The label of a node should be actor or movie name depending on the type of the node. Edges should be styled appropriately.

In addition, when the user right clicks on an actor node, a context menu should appear with the sole content of “Show movies”. When the user selects this option, all movies in which that actor played should be added to the current graph if not already in the graph. In the same way, when a user right clicks on a movie node, a context menu should appear with “Show actors”, and upon selection, all actors of that movie, if not already in the graph, should be added to the current graph. Whenever the graph is extended with such context menu operations, an incremental (randomize option of the layout algorithm should be set to false) layout should be performed to adjust the layout.

Enrich the App

Feel free to use your imagination to enrich this application with different use cases, queries to the database or context menu options. Remember the main goal is to let the movie lovers explore this movie database visually.

Suggested Extensions to Use

We recommend using the following extensions for this application:

- Use [this](#) for context menus. Make sure the menu items are context-sensitive (e.g., include “Show actors” in the context menu of a movie but not “Show movies”).
- Use [this](#) for highlighting neighbors or newly integrated graph objects (e.g., when new persons are brought from the database on “Show actors” of a movie).
- Use [this](#) to initially position newly integrated graph objects and apply an incremental layout (randomize: false) to keep the overall layout.

Submission

You are expected to share your implementation as a GitHub repository and let us know when finished. Please add a deployment guide to the Readme of this repository so that we can build and test your code.